FIELD LAB MANUAL

Elastic Stack on Ubuntu Virtual Machines

Three-node Elasticsearch cluster + Kibana using Virtual Machine Manager (KVM/libvirt)

Labs included

LAB	EXERCISE	CORE RESULT
L1	Cluster deployment and outage	Node membership, shard placement, availability and recovery
L2	Security boundaries	Operator/analyst roles, Spaces, index access and optional FLS
Q1	Search basics	Service-log ingestion, KQL, time filtering and document inspection

Intended audience: Linux/cloud operations engineers, administrators and instructor-led trainees.

Platform: Ubuntu Server guests on an Ubuntu host running Virtual Machine Manager; Elastic 9.x Debian packages.

Edition: 29 October 2026 | **Mode:** Guided, self-managed training lab

LAB SAFETY Use an isolated private network, synthetic logs and unique test passwords. Do not expose Elasticsearch or Kibana to the public Internet. This manual is for training, not production hardening.

How to use this manual

Follow the sections in order. Existing VMs can be reused; do not rebuild working nodes. Replace every sample IP with the address shown inside your own VM. Console queries beginning with **GET**, **PUT** or **POST** are for **Kibana Dev Tools**, not the Ubuntu terminal.

SECTION	PURPOSE	REQUIRED EVIDENCE
A. Prepare	VM sizing, networking, prerequisites	IP / VM inventory
B. Deploy	Three Elasticsearch nodes plus Kibana	Healthy cluster and UI
L1	Shards, controlled outage, recovery	Before/during/after screenshots
L2	Roles, users, Spaces, FLS when licensed	Allowed/denied test matrix
Q1	Logs, data view, KQL queries	Three saved-query results
Submission	Checklists and sign-off	Final evidence folder

Learning outcomes

- Explain three-node cluster topology and the role of master-eligible/data nodes.
- Install secure Elastic packages, join nodes by enrollment token and inspect health.
- Demonstrate continued reads after a single node failure and discuss replica rebuilding.
- Apply least-privilege index and Kibana Space authorization.
- Use KQL, time filters and document inspection to answer operational questions.

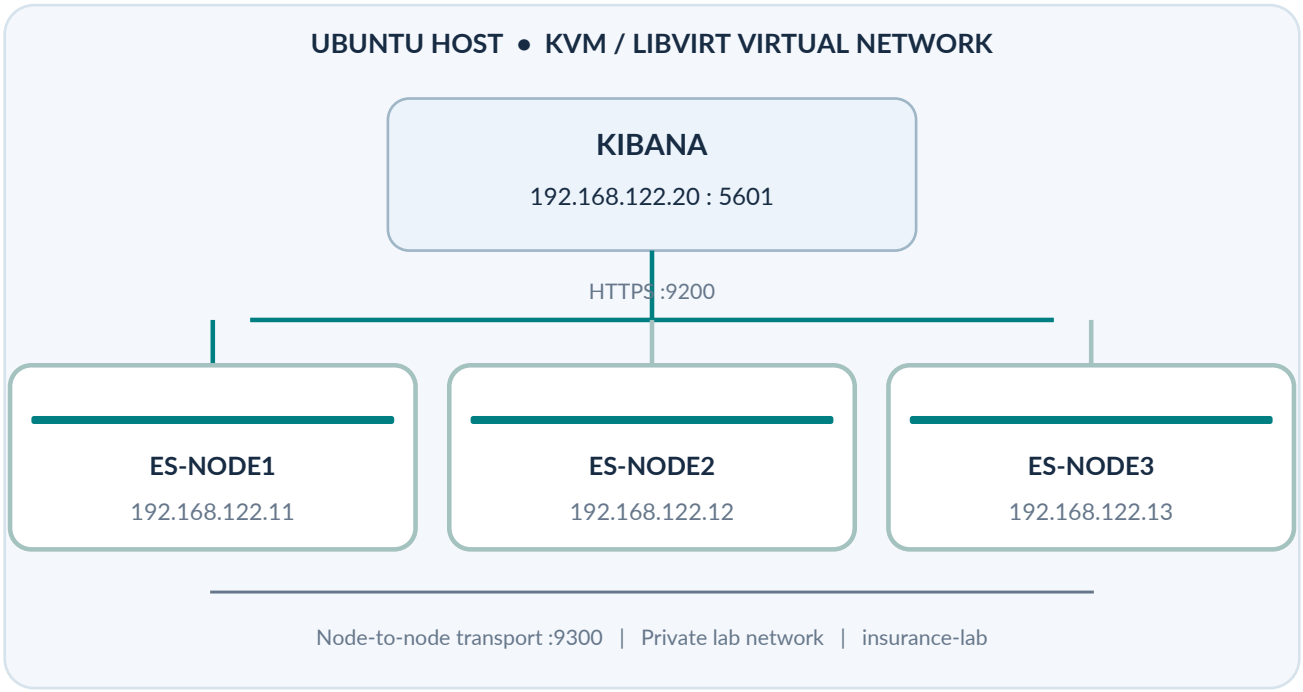
Before beginning

Assumption You have an Ubuntu physical machine with Virtual Machine Manager installed and three Ubuntu Server VMs (or enough capacity to create them). The fourth Kibana VM is optional: Kibana can instead run on the host.

- ☐ Record hostname, IP address, OS release and VM RAM for every VM.
- ☐ Confirm Internet access for APT package download and private inter-VM connectivity.
- ☐ Reserve the training VM addresses through libvirt DHCP or equivalent if repeatability matters.

A. Prepare the Ubuntu / KVM environment

A1. Topology and capacity



Illustrative topology only: IP addresses are examples from the default libvirt NAT subnet.

HOSTNAME	EXAMPLE IP	CPU	RAM	DISK
es-node1	192.168.122.11	2 vCPU	3 GB	25 GB
es-node2	192.168.122.12	2 vCPU	3 GB	25 GB
es-node3	192.168.122.13	2 vCPU	3 GB	25 GB
kibana	192.168.122.20	2 vCPU	3 GB	25 GB

Sizing These are classroom starter allocations, not production sizing. Prefer a host with 24 GB+ memory. On a 16 GB host, reduce allocations carefully, monitor swapping/OOM and consider placing Kibana on the host.

A2. Training configuration worksheet

ITEM	WRITE ACTUAL VALUE
Ubuntu host physical memory	_____
Libvirt network / subnet	_____
Elasticsearch version	_____
Kibana version (must match)	_____
Cluster name	insurance-lab
Node addresses	N1: _____ N2: _____ N3: _____
Kibana URL	_____

A3. Verify Virtual Machine Manager and networking

Execute on the Ubuntu **physical host**. If Virtual Machine Manager is already installed, the following command is still safe to run.

HOST COMMANDS

```
sudo apt update
sudo apt install -y virt-manager libvirt-daemon-system \
  libvirt-clients qemu-kvm
sudo virsh net-list --all
sudo virsh net-dhcp-leases default
```

In Virtual Machine Manager: open each VM, select its NIC device, and attach it to the same libvirt **default NAT** network (or your existing private bridged network). Start each VM.

A4. Set unique hostnames and find addresses

On each guest, change the hostname argument to match the VM. Record the actual IP address; DHCP leases may not match the diagram.

RUN IN EACH GUEST, CHANGING HOSTNAME

```
sudo hostnamectl set-hostname es-node1
hostnamectl
ip -4 -br addr
hostname -I
```

A5. Confirm connectivity

FROM ES-NODE1; REPLACE PLACEHOLDERS

```
ping -c 3 <NODE2_IP>
ping -c 3 <NODE3_IP>
ping -c 3 <KIBANA_IP>
```

All Elasticsearch VMs need TCP **9300** to connect to one another. Allow **9200** only from trusted lab clients and **5601** only from the lab host to Kibana; keep the cluster isolated. Use the actual subnet rather than allowing arbitrary Internet access.

A6. Elasticsearch OS prerequisite

Run on each Elasticsearch VM. If the value is less than 1048576, set it persistently:

```
sysctl vm.max_map_count
echo "vm.max_map_count=1048576" | \
  sudo tee /etc/sysctl.d/99-elasticsearch.conf
sudo sysctl --system
sysctl vm.max_map_count
```

Gate A Continue only when all three VMs can reach one another, hostnames are unique, and vm.max_map_count is at least 1048576.

B. Deploy Elasticsearch and Kibana

B1. Add Elastic 9.x APT repository

Run on each of the three Elasticsearch VMs. Run the same repository steps on the Kibana VM before installing Kibana.

```
sudo apt update
sudo apt install -y curl wget gnupg ca-certificates
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/elasticsearch-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] \
https://artifacts.elastic.co/packages/9.x/apt stable main" \
  | sudo tee /etc/apt/sources.list.d/elastic-9.x.list
sudo apt update
```

Version discipline Use the same exact version across all Elasticsearch nodes and Kibana. The 9.x repository tracks new releases; on long-running courses consider pinning all guests to the same apt package version.

B2. Install ES Node 1

```
sudo apt install -y elasticsearch
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Uncomment or set these values, without duplicating existing keys. Retain generated TLS and security settings.

ELASTICSEARCH.YML ON NODE 1

```
cluster.name: insurance-lab
network.host: 0.0.0.0
transport.host: 0.0.0.0
```

For a brand-new auto-configured first node, do not invent or duplicate initial bootstrap settings. Keep any generated settings until the cluster has formed. Network exposure beyond this private lab needs tighter binding and firewall rules.

START NODE 1

```
sudo systemctl daemon-reload
sudo systemctl enable --now elasticsearch
sudo systemctl status elasticsearch --no-pager
```

B3. Set elastic password and verify Node 1

SET A STRONG PASSWORD ON ES-NODE1

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password \
  -u elastic -i
```

Store the password privately; do not paste it into screenshots or the final evidence report.

CHECK REST API ON NODE 1

```
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u elastic https://localhost:9200
```

Expected: JSON with cluster name **insurance-lab**, node name and Elasticsearch version.

B4. Enroll Node 2

Generate a token on **es-node1**. Enrollment tokens last about 30 minutes.

ON EXISTING NODE 1

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token \
  -s node
```

On **es-node2**, install Elasticsearch via APT if not done already, but **do not start the service before enrolling**.

ON NODE 2

```
sudo apt install -y elasticsearch
sudo /usr/share/elasticsearch/bin/elasticsearch-reconfigure-node \
  --enrollment-token "PASTE_TOKEN_HERE"
```

Edit **/etc/elasticsearch/elasticsearch.yml**; ensure the following settings exist and keep the enrollment-generated discovery and TLS settings:

NODE 2 CONFIGURATION

```
cluster.name: insurance-lab
network.host: 0.0.0.0
transport.host: 0.0.0.0
```

START NODE 2

```
sudo systemctl enable --now elasticsearch
sudo systemctl status elasticsearch --no-pager
```

If enrollment fails Check that Node 1 is reachable from Node 2, token has not expired, Node 2 has never started as a standalone cluster, and generated security configuration was not deleted. Use a fresh VM snapshot if the enrollment state is already inconsistent.

B5. Enroll Node 3 and finalize discovery

Generate **a new** token on Node 1. On Node 3, install but do not start Elasticsearch before running reconfiguration.

NODE 1: FRESH ENROLLMENT TOKEN

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token \
  -s node
```

NODE 3: CONFIGURE BEFORE FIRST START

```
sudo apt install -y elasticsearch
sudo /usr/share/elasticsearch/bin/elasticsearch-reconfigure-node \
  --enrollment-token "PASTE_NEW_TOKEN_HERE"
```

Set **cluster.name**, **network.host** and **transport.host** as on Node 2, and then start the service.

START NODE 3

```
sudo systemctl enable --now elasticsearch
```

Discovery configuration for stable restarts

Once all three nodes have joined, edit **/etc/elasticsearch/elasticsearch.yml** on **each node**. Keep existing generated transport TLS settings and add/update:

REPLACE ALL EXAMPLE IP ADDRESSES

```
discovery.seed_hosts:
  - "192.168.122.11:9300"
  - "192.168.122.12:9300"
  - "192.168.122.13:9300"
```

Comment out or remove **cluster.initial_master_nodes** if present, **only after initial cluster formation**. It must not be set again on ordinary restarts. The first node requires seed hosts added manually.

Verify Elasticsearch node membership

From Node 1, use the local REST endpoint with its HTTP CA certificate:

ON NODE 1

```
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u elastic "https://localhost:9200/_cat/nodes?v"
```

Pass: exactly three node rows appear and each node belongs to **insurance-lab**.

B6. Install and enroll Kibana

On the **kibana VM**, first install the Elastic repository as in B1, then:

KIBANA INSTALLATION

```
sudo apt install -y kibana
sudo nano /etc/kibana/kibana.yml
```

For private-lab browser access, set:

KIBANA.YML

```
server.host: "0.0.0.0"
server.port: 5601
```

START KIBANA

```
sudo systemctl daemon-reload
sudo systemctl enable --now kibana
sudo systemctl status kibana --no-pager
```

On **es-node1**, generate a Kibana enrollment token:

ON ES NODE 1

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token \
  -s kibana
```

From a browser on the Ubuntu host, open **http://<KIBANA_IP>:5601**. Paste the enrollment token, provide the verification code if prompted, and sign in as **elastic** with your password.

Alternative If host memory is constrained, install Kibana directly on the Ubuntu host. Use localhost:5601 there; configure routing/firewall so the host can reach the nodes over TCP 9200.

B7. Verify from Kibana Dev Tools

Open **Dev Tools > Console** (use Kibana global search if navigation differs). Execute each request individually:

KIBANA CONSOLE (NOT BASH)

```
GET _cat/nodes?v
GET _cat/master?v
GET _cluster/health?pretty
GET _cat/allocation?v
```

Pass: three nodes listed; one elected master; health eventually green when all primaries and replicas are assigned.

- ☐ Screenshot: Kibana Welcome/Console and output of _cat/nodes.
- ☐ Record exact Elastic version and number of nodes.

L1. Cluster deployment, availability and recovery

Objective. Demonstrate node membership, primary/replica shard placement, cluster health, continued access during one controlled node outage and successful recovery. Do not stop two nodes together.

L1.1. Create a replicated index

Run in Kibana Dev Tools. Two primaries and one replica per primary create **four shard copies** in total.

CONSOLE: CREATE LAB INDEX

```
PUT service-logs-lab
{
  "settings": {
    "number_of_shards": 2,
    "number_of_replicas": 1
  },
  "mappings": {
    "properties": {
      "@timestamp": { "type": "date" },
      "service": { "properties": {
        "name": { "type": "keyword" } } },
      "log": { "properties": {
        "level": { "type": "keyword" } } },
      "http": { "properties": {
        "response": { "properties": {
          "status_code": { "type": "integer" } } } } },
      "message": { "type": "text" },
      "customer": { "properties": {
        "email": { "type": "keyword" } } }
    }
  }
}
```

L1.2. Capture baseline

CONSOLE

```
GET _cat/nodes?v
GET _cat/shards/service-logs-lab?v
GET _cluster/health/service-logs-lab?pretty
```

Expected: four shard copies in STARTED state if allocation permits. The replica for a primary cannot be allocated on the same node as its primary.

> **Before outage** Load 10 sample documents using Q1.1, then return here. A count check makes the resilience demonstration more meaningful.

L1.3. Simulate one node outage

Choose **es-node3** (not the current elected master if you want a simpler first demonstration). Record the baseline count, and then on that Ubuntu VM run:

TERMINAL ON THE SELECTED NODE

```
sudo systemctl stop elasticsearch
```

From Kibana while Node 1/2 are still available, execute:

KIBANA CONSOLE

```
GET _cat/nodes?v
GET _cluster/health/service-logs-lab?pretty
GET _cat/shards/service-logs-lab?v
GET service-logs-lab/_count
```

L1.4. Explain the result

OBSERVATION	INTERPRETATION
Cluster has two nodes	The stopped node is not participating in cluster operations.
An affected primary disappears	A healthy replica on another node may be promoted to primary.
Yellow health during recovery	Primaries are active, but one or more replicas are unassigned.
Green after reallocation	All required copies are allocated across remaining eligible nodes.
Document count remains 10	Expected if enough surviving copies exist and there were no unreplicated writes.

The exact health transition and placement depend on actual allocation, disk availability and timing. Do not assert that health must be yellow or green at a particular second.

L1.5. Recover the stopped VM service

ON STOPPED NODE

```
sudo systemctl start elasticsearch
sudo systemctl status elasticsearch --no-pager
```

CONSOLE

```
GET _cat/nodes?v
GET _cat/shards/service-logs-lab?v
GET _cluster/health/service-logs-lab?wait_for_status=green&timeout=120s
```

L1.6. Evidence to submit

- ☐ Before: 3 nodes, initial health, shard placement, document count.
- ☐ During: 2 nodes, health, shard placement, unchanged searchable document count.
- ☐ After: 3 nodes again, shard recovery and eventual healthy state.
- ☐ One paragraph explaining replica promotion and why a two-node failure is outside this exercise.

L2. Security boundaries: roles, users and Spaces

Objective. Apply least privilege for two personas. Space permissions govern Kibana content access; index privileges govern Elasticsearch data. Neither replaces the other.

L2.1. Create Kibana Spaces

Sign in as **elastic**. Use global search to open **Spaces**. Create two spaces with the **Classic** solution view (easy for cross-feature training):

SPACE	URL ID	USE
Lab Operations	lab-operations	Operator index management and testing
Lab Analytics	lab-analytics	Discover, log views and saved queries

Create the service-log data view later in **Lab Analytics** as an administrator or as an account with the necessary Data View management privileges. A data view belongs to its Space.

L2.2. Create operator role and user

Open **Roles** from Stack Management/global search; click **Create role**.

SETTING	VALUE
Role name	lab_operator
Cluster privileges	monitor
Index names	service-logs-lab
Index privileges	all (for the dedicated lab index only)
Kibana Spaces	Lab Operations and Lab Analytics
Kibana privilege	All in both selected Spaces

Create native user **operator1** with only **lab_operator**. It can write to the lab index, but should not have cluster security administration privileges.

L2.3. Create analyst role and user

SETTING	VALUE
Role name	lab_analyst
Cluster privileges	None
Index names	service-logs-lab
Index privileges	read, view_index_metadata
Kibana Spaces	Lab Analytics only
Kibana privilege	Read

Create native user **analyst1** with only **lab_analyst**. Do not assign broad built-in roles such as superuser or kibana_admin to either lab persona.

L2.4. Test positive and negative access

Sign out between roles or use separate private browser sessions. Check the Spaces selector, Discover and a direct API write test. The read-only analyst may not be able to save a query; save instructor evidence with the administrator/operator session.

TEST	OPERATOR1	ANALYST1
Search service-logs-lab	PASS / allowed	PASS / allowed
Create a document in service-logs-lab	PASS / allowed	DENIED (403)
Open Lab Operations Space	PASS / allowed	DENIED / hidden
Open Lab Analytics Space	PASS / allowed	PASS / allowed
Modify security roles	DENIED	DENIED

Run the following from an Elasticsearch VM terminal. Authenticate as **analyst1** and enter the password when prompted:

EXPECTED: HTTP 403 / SECURITY_EXCEPTION

```
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u analyst1 -X POST \
  "https://localhost:9200/service-logs-lab/_doc" \
  -H "Content-Type: application/json" \
  -d '{"message":"unauthorized write test"}'
```

To record a deterministic response code, optionally append **-i** to the curl command. A failed write must not increase the indexed document count.

L2.5. Optional field-level security (license dependent)

If your license/trial offers Field Level Security, edit **lab_analyst** under index privileges. Grant all needed fields but deny **customer.email**. Confirm it is excluded from analyst reads of matching documents.

Field-level and document-level security should be used with read-only index privileges. If another role grants unrestricted access to the same index, the restriction may be bypassed due to combined grants.

Licensing Do not fail the basic lab if the Field Level Security control is unavailable. Record "Not tested - feature not enabled under the current license" as the outcome.

L2.6. Evidence to submit

- Role settings screenshots for lab_operator and lab_analyst.
- Spaces access screenshots for both users.
- Successful analyst read and denied analyst write (403).
- FLS test result, or license-based exception recorded.

L2.7. Interpret why a test was denied

ACTION	SECURITY BOUNDARY	EXPLAIN IN REPORT
Analyst can read service logs	Index privileges	read + view_index_metadata
Analyst cannot write documents	Index privileges	No create/index/write grant
Analyst cannot open Operations	Kibana Space privilege	Only Analytics Space permitted
Analyst cannot see customer.email	Field-level privileges	Only when FLS is enabled and correctly applied

Discussion: Would hiding a Space also stop a direct REST API query? **No.** Kibana Spaces protect Kibana features/content, while Elasticsearch index privileges protect documents. State which control blocked each negative test.

Q1. Search basics: ingest synthetic service logs

Objective. Load a small, known service-log dataset, search it using KQL and time filters, inspect individual documents and save evidence for three operational questions.

Q1.1. Generate ten timestamped log events

Run on **es-node1**. The generator writes newline-delimited JSON for the Elasticsearch Bulk API with timestamps relative to execution time, keeping the last-24-hours Discover filter reliable.

TERMINAL: CREATE TEST DATASET

```
python3 - <<'PY' > /tmp/service-logs.ndjson
import json
from datetime import datetime, timezone, timedelta
rows = [
  ("policy-api",  "INFO",  200, "Policy activated"),
  ("claims-api",  "ERROR", 500, "Claim submission failed"),
  ("claims-api",  "WARN",  429, "Claim validation slow"),
  ("payment-api", "ERROR", 504, "Payment gateway timeout"),
  ("payment-api", "INFO",  200, "Payment completed"),
  ("claims-api",  "INFO",  201, "Claim submitted"),
  ("policy-api",  "ERROR", 503, "Policy lookup failed"),
  ("claims-api",  "ERROR", 500, "Claim assessment failed"),
  ("policy-api",  "WARN",  429, "Renewal check slow"),
  ("payment-api", "INFO",  200, "Refund completed")
]
for i,(svc,level,status,msg) in enumerate(rows):
  doc={
    "@timestamp": (datetime.now(timezone.utc)
      - timedelta(minutes=i)).isoformat(),
    "service":{"name":svc},
    "log":{"level":level},
    "http":{"response":{"status_code":status}},
    "message":msg,
    "customer":{"email":"customer@example.test"}
  }
  print(json.dumps({"index":{"_index":"service-logs-lab"}}))
  print(json.dumps(doc))
PY
```

Q1.2. Ingest logs and check document count

TERMINAL: BULK INDEX 10 DOCUMENTS

```
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u elastic -H "Content-Type: application/x-ndjson" \
  -X POST "https://localhost:9200/_bulk?refresh=true" \
  --data-binary @/tmp/service-logs.ndjson
```

Inspect the JSON response for **"errors": false**; then in Kibana Console:

CONSOLE VERIFICATION

```
GET service-logs-lab/_count
GET service-logs-lab/_search?size=2
```

Expected: 10 documents after a single successful load. Re-running the bulk load creates another 10 documents because auto-generated IDs are used; if you rerun, delete/recreate the lab index or adjust expected counts.

Q1.3. Create a data view

As an admin/operator in the **Lab Analytics** Space, open **Data Views** using Kibana global search. Create:

SETTING	VALUE
Display name	Service Logs
Index pattern	service-logs-lab
Time field	@timestamp
Space	Lab Analytics

Open **Discover**, choose **Service Logs**, select the **Last 24 hours** time range, and confirm the query bar uses **KQL** (not Lucene). If you generated the logs more than 24 hours ago, widen the time range.

Q1.4. Three KQL tasks

QUESTION	KQL FILTER	EXPECTED
1. How many ERROR logs?	`log.level: ERROR`	4 results
2. How many ERROR logs in claims-api?	`service.name: "claims-api"` `AND log.level: ERROR`	2 results
3. Payment service, server errors?	`service.name: "payment-api"` `AND http.response.status_code >= 500`	1 result

For each query, open one row and inspect **message**, **@timestamp**, **service.name**, **log.level** and **http.response.status_code**. Save the query/Discover session using an account with the necessary Kibana permissions.

Q1.5. Time-filter and document-inspection evidence

Each answer needs more than a screenshot of the query text. Show the selected data view, time range, number of hits, KQL filter and a matching document.

EVIDENCE	RECORD / SCREENSHOT
Search 1 - Errors	4 hits; example message: Claim submission failed
Search 2 - Claims errors	2 hits; example message: Claim assessment failed
Search 3 - Payment server errors	1 hit; message: Payment gateway timeout
Time filter	Last 24 hours; timestamp field @timestamp
Document inspection	Expanded record showing fields and values

Expected data quality check

SERVICE	TOTAL EVENTS	ERROR EVENTS
policy-api	3	1
claims-api	4	2
payment-api	3	1
TOTAL	10	4

Optional extension tasks

- Filter only HTTP 5xx across all three services using http.response.status_code >= 500.
- Use a time range of Last 15 minutes immediately after ingestion and explain missing hits if the exercise runs late.
- Run the same filter as operator1 and analyst1 to compare search authorization.
- Explain the difference between KQL filtering and an Elasticsearch REST API search request.

Q1.6. Evidence to submit

- ☐ Bulk indexing response with errors:false and count 10.
- ☐ Service Logs data view showing @timestamp as time field.
- ☐ Three searches with KQL, Last 24 hours and expected counts.
- ☐ One expanded document per search.
- ☐ Saved query/Discover artifact names and screenshots.

Troubleshooting decision guide

SYMPTOM	CHECK / NEXT ACTION
Elasticsearch fails to start	Use systemctl status elasticsearch and journalctl -u elasticsearch -n 100. Check vm.max_map_count, YAML and memory.
Only one or two nodes visible	Verify discovery.seed_hosts, ports 9300, cluster.name, enrollment and transport TLS. Inspect logs on missing node.
First node cannot restart	After forming a multi-node cluster, add discovery.seed_hosts to the first node and remove stale initial_master_nodes.
Kibana displays setup page indefinitely	Inspect journalctl -u kibana -n 100, recreate expired Kibana enrollment token and check port 9200 reachability.
Kibana does not open in browser	Check server.host, VM IP, port 5601, private firewall and VM NIC mode.
Yellow index health	Inspect _cat/shards, disk watermarks, node count and replica allocation. Yellow means some replica copies unassigned.
Analyst unexpectedly writes successfully	Inspect every assigned role; privilege grants combine. Remove broad roles and retry with a clean user.
Discover shows 0 documents	Check _count, selected data view, @timestamp and time picker. Refresh data view fields as required.
Duplicate query totals	Bulk API used auto-generated IDs. Recreate the index or start fresh before repeating the exercise.

Useful commands

LINUX TERMINALS

```
sudo journalctl -u elasticsearch -n 100 --no-pager
sudo journalctl -u kibana -n 100 --no-pager
sudo systemctl status elasticsearch --no-pager
ss -lntp | grep -E ":(9200|9300|5601)"
```

KIBANA DEV TOOLS

```
GET _cluster/health?pretty
GET _cat/nodes?v
GET _cat/shards?v
GET _cluster/allocation/explain
```

Recovery discipline For the controlled outage, stop and start only the Elasticsearch service. Do not delete /var/lib/elasticsearch, regenerate a cluster UUID or modify bootstrap settings to force a join.

Evidence package and trainer evaluation

Create a folder named **Elastic_Lab_Evidence**. Store screenshots as PNG/JPG, a short lab report, and text exports of non-secret CLI output. Redact passwords, enrollment tokens, authorization headers and real customer information.

Suggested naming convention

```
01_Topology_and_VM_Inventory.png
02_L1_Nodes_Initial.png
03_L1_Shards_Initial.png
04_L1_Node_Stopped.png
05_L1_Recovery.png
06_L2_Operator_Role.png
07_L2_Analyst_Role.png
08_L2_Access_Denied_403.png
09_Q1_Error_KQL.png
10_Q1_Claims_KQL.png
11_Q1_Payment_KQL.png
12_Q1_Document_Inspection.png
```

Lab completion checklist

- ☐ IP inventory, OS version and hardware allocation recorded.
- ☐ Kibana connects securely to a three-node Elasticsearch cluster.
- ☐ Cluster health / elected master / shard placement recorded.
- ☐ One node stopped, data count validated, and node recovered.
- ☐ Operator and analyst users tested across both Spaces.
- ☐ Analyst denied an index write with a 403 response.
- ☐ FLS tested if available, or documented as not licensed.
- ☐ Ten events ingested and all three KQL results captured.

Trainer scoring rubric

AREA	POINTS	EVALUATION
Environment and topology	20	Accurate inventory, reachable nodes, Kibana works
L1 cluster and recovery	30	Membership, replicas, outage, read test, recovery
L2 authorization	25	Least privilege and positive/negative tests
Q1 searches	20	10 records, 4/2/1 counts, time filters, inspection
Documentation quality	5	Named, readable evidence and sound explanations
TOTAL	100	Recommended pass threshold: 75

Learner report template

Participant: _____

Date / batch: _____

Ubuntu host hardware: _____

Elasticsearch / Kibana version: _____

Lab L1 - Observations

METRIC	BEFORE OUTAGE	DURING OUTAGE	AFTER RECOVERY
Node count	_____	_____	_____
Index health	_____	_____	_____
Primary shards	_____	_____	_____
Replica state	_____	_____	_____
Document count	_____	_____	_____

Explain replica availability and recovery:

_____

_____

_____

Lab L2 - Authorization findings

TEST CASE	ACTUAL RESULT	EVIDENCE FILE
Operator write	_____	_____
Analyst read	_____	_____
Analyst write denied	_____	_____
Analytics Space access	_____	_____
Operations Space denied	_____	_____
FLS / licensing outcome	_____	_____

Q1 - Search results

QUERY	EXPECTED	ACTUAL	SCREENSHOT
All ERROR logs	4	_____	_____
Claims ERROR logs	2	_____	_____
Payment HTTP 5xx	1	_____	_____

Official references and scope

Use the following official Elastic documentation to validate current versions, enrollment, security and query syntax. Navigation labels may vary slightly between Elastic minor releases.

01. Debian package installation - Elasticsearch
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-with-debian-package

02. Debian package installation - Kibana
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-kibana-with-debian-package

03. Discovery and cluster formation settings
https://www.elastic.co/docs/reference/elasticsearch/configuration-reference/discovery-cluster-formation-settings

04. Linux virtual memory setting
https://www.elastic.co/docs/deploy-manage/deploy/self-managed/vm-max-map-count

05. Manage Kibana Spaces
https://www.elastic.co/docs/deploy-manage/manage-spaces

06. Kibana role management
https://www.elastic.co/docs/deploy-manage/users-roles/cluster-or-deployment-auth/kibana-role-management

07. Field and document level access
https://www.elastic.co/docs/deploy-manage/users-roles/cluster-or-deployment-auth/controlling-access-at-document-field-level

08. Kibana Query Language reference
https://www.elastic.co/docs/reference/query-languages/kql

Scope The lab demonstrates self-managed cluster behavior and access boundaries. A production deployment also needs capacity testing, backups, lifecycle retention, full TLS for end-user access, monitoring, operational runbooks, secrets management and disaster recovery.

Completion statement

The learner is successful when all three nodes participate in one cluster, replicas preserve search access during the one-node outage, space/index permissions yield the planned allow/deny outcomes, and three KQL queries return the expected results from the synthetic dataset.